

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS)

PROJECT: RESTAURANT MANAGEMENT SYSTEM

Document Version = 1.0

Date: July 10, 2026

Chapter 1: Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document serves as a reference point for all stakeholders of the Restaurant Management System (RMS), capturing both functional and non-functional requirements. Its purposes include:

- **Capture Requirements:** Documenting the complete reservation-to-payment workflow of a restaurant visit.
- **Communication Tool:** Facilitating clear understanding among the restaurant's Manager, Waiters, Customers, and the development team.
- **Basis for Agreement:** Establishing mutual understanding of the business rules that govern the system, such as automatic table allocation.
- **Guidance for Development:** Directing design and development activities, including implementation in Spring Boot.
- **Basis for Validation and Verification:** Serving as the reference for testing against the defined acceptance criteria.
- **Change Management:** Managing changes to requirements as the project evolves.
- **Documentation of Design Rationale:** Recording why responsibilities are split in the way they are (Section 1.3).

1.2 Scope

The Restaurant Management System automates the complete lifecycle of a dine-in customer visit: booking a table, automatic allocation of the best-fit table, seating, placing one or more food orders, billing, payment confirmation, and releasing the table for the next customer. Three user roles are in scope — Customer, Waiter, and Manager — each with clearly separated responsibilities (Section 2.3, Security). Delivery, takeaway, and multi-branch operation are out of scope for this version.

The Reservation is the central entity of the system: every table of allocation, order, and payment exists in relation to a Reservation. This is illustrated in the workflow below and reflected throughout Chapter 2.

1.3 Design Approach

The system is deliberately organized as a set of single-responsibility components rather than a generic CRUD application: Reservation manages the customer's visit, RestaurantTable manages the physical table, Order manages food requests, Payment manages billing, Waiter manages restaurant-floor operations, and Manager manages administration. This separation, carried through to the Functional Requirements in Chapter 2 and the diagrams in Sections 2.4–2.5, keeps the system easy to understand, test, and extend.

1.4 System Workflow

Every requirement in this document supports one step of the following end-to-end workflow; from the moment a customer arrives to the moment their table is reassigned to the next waiting customer.

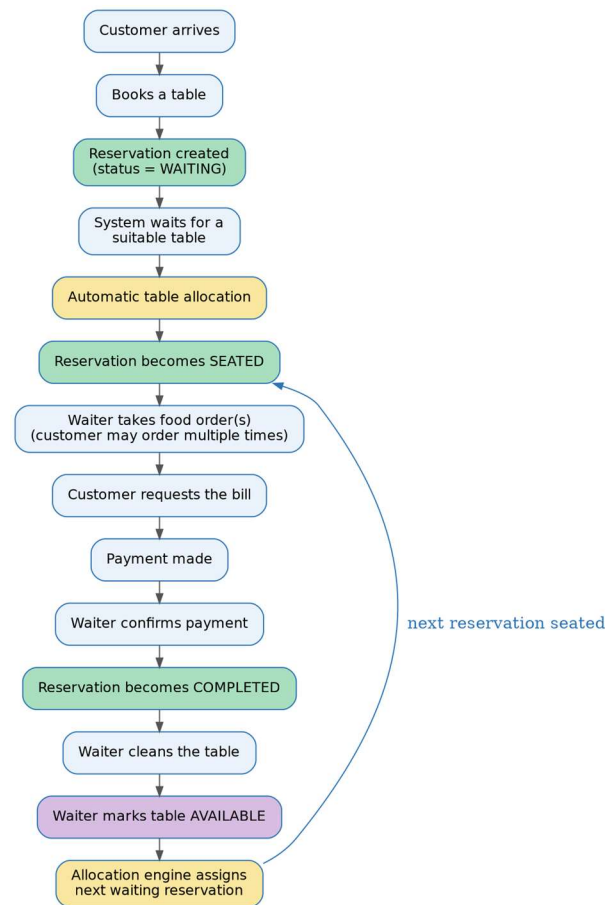


Figure 1: End-to-End Reservation Workflow

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
RMS	Restaurant Management System
RBAC	Role-Based Access Control — restricting system access based on the logged-in user's role
WAITING	Reservation status before a table has been allocated
SEATED	Reservation status once a table has been allocated and the customer is seated
COMPLETED	Reservation status once the assigned waiter has confirmed payment
Allocation Engine	The business logic component that assigns the best-fit waiting reservation to a table as soon as it becomes available
Best-fit Allocation	Assigning a freed table to the waiting reservation whose party size is the closest match without exceeding capacity
ER Diagram	Entity Relationship Diagram, depicting the data model (Section 2.4)

Chapter 2: Specific Requirements

2.1 Functional Requirements

Functional requirements describe what the system should do. These are organized below by module, following the workflow in Section 1.4. Role restrictions noted under each module (who may perform each action) are enforced via the Role-Based Access Control model defined in Section 2.3 (Security).

2.1.1 Customer Management

Stores the identity data needed to create a reservation. This module is intentionally minimal — a customer interacts with the rest of the system only through a Reservation, never directly with tables or orders.

Features:

- Store Customer (Name, Phone Number, Email — optional).
- View own Reservation status and assigned table.
- A customer may create multiple reservations over time.

2.1.2 Reservation Management

The central module of the system. Every customer must create a reservation — with mandatory party size and booking time — before any other activity (ordering, payment) can occur. A reservation always starts with status WAITING, moves to SEATED once a table is allocated (Section 2.1.4), and becomes COMPLETED once payment is confirmed (Section 2.1.8).

Features:

- Create Reservation (Customer action; party size mandatory, email optional).
- View Reservation status and history.
- Update Reservation status (system/waiter-driven, per Section 2.2 rules).
- Cancel Reservation.

2.1.3 Table Management

Maintains the restaurant's physical seating. A table does not reference customers directly — Reservations reference tables, not the other way around.

Features:

- Add / Update Table (Table Number, Capacity) — Manager only.
- View Table status (AVAILABLE, OCCUPIED, or allocated).
- Update Table status — Waiter only (Section 2.1.9).

2.1.4 Automatic Table Allocation

Whenever a table becomes AVAILABLE, the system searches the waiting reservations and assigns the table to the best-fit match — the reservation whose party size is closest to the table's capacity without exceeding it — rather than a random or strictly first-come assignment. This maximizes table utilization.

Features:

- Example: Tables of capacity 2, 4, and 6 exist.
The waiting queue holds reservations of 5, 4, 2, and 1 people. When the capacity-4 table frees up, the engine checks $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ and assigns it to the 4-person reservation (or the 2-person reservation if no 4-person reservation is waiting).
- The customer cannot manually select a table; allocation is fully automatic.
- On successful allocation, the linked Reservation status changes from WAITING to SEATED.

2.1.5 Waiter Management

Every waiter is assigned a fixed set of tables by the Manager (e.g., Waiter A $\rightarrow$ Tables 1–4). A waiter's access and duties are scoped to their assigned tables only.

Features:

- Assign Waiter to Table(s) — Manager only.
- View Waiter's assigned tables and active reservations.

2.1.6 Menu Management

Maintains the list of available food items. Only the Manager can modify the menu; Customers and Waiters have view-only access, consistent with Section 2.3.

Features:

- Add / Update / Remove Menu Item (Name, Category, Price, Availability) — Manager only.
- View / Browse Menu — Customer and Waiter.
- Mark Item Available / Out of Stock — Manager only.

2.1.7 Order Management

Orders can only be placed once a reservation is SEATED. A single reservation may have multiple orders over the course of the visit (e.g., drinks, then food, then dessert), and each order may contain multiple items.

Features:

- Create Order against a SEATED Reservation — assigned Waiter only.
- Add Items to Order (Menu Item + Quantity).
- View Order(s) for a Reservation.

2.1.8 Payment Management

Once dining is complete, a bill is generated from the reservation order(s) and the customer pays. Payment is not finalized automatically — it requires explicit confirmation.

Features:

- Generate Bill from Reservation's Order(s).
- Record Payment — Customer action.
- Confirm Payment — assigned Waiter only; this transitions the Reservation to COMPLETED.

2.1.9 Table Release

The table is not freed automatically when payment is confirmed — it remains occupied until the waiter has physically cleared and prepared for it.

Features:

- Mark Table AVAILABLE — assigned Waiter only, after cleaning.
- Freeing a table automatically triggers Automatic Table Allocation (Section 2.1.4) for the next waiting reservation.

2.2 Business Rules

The following rules apply across the modules above and govern valid state transitions.

- A reservation always starts in WAITING status; party size is mandatory; email is optional.
- Only tables marked AVAILABLE are eligible for allocation, and allocation always follows the best-fit logic in Section 2.1.4 — never manual selection.
- Orders may only be created once a reservation is SEATED, and only by that table's assigned waiter.
- A reservation may have multiple orders; an order may have multiple items.
- A reservation becomes COMPLETED only when the assigned waiter confirms payment — never automatically.
- A table becomes AVAILABLE only when the assigned waiter marks it so — never automatically after payment — which then triggers the next allocation.

2.3 Non-Functional Requirements

Non-functional requirements describe how the system should perform — its quality attributes, performance expectations, and operational characteristics.

Performance

Table allocation must complete within 1–2 seconds of a table becoming AVAILABLE, so waiting customers are seated promptly. Standard operations (order entry, status updates) should respond within 2 seconds under normal load.

Security

Access to every module in Section 2.1 is restricted by Role-Based Access Control (RBAC) across exactly three roles — Customer, Waiter, and Manager. The table below defines what each role is authorized to access; this is the authorization model and is considered finalized. The authentication mechanism used to verify a user's identity before this authorization applies (e.g., session-based login, JWT, OAuth2) is a separate, not-yet-finalized decision to be made in the technical design phase. Regardless of the mechanism chosen, passwords must be hashed and all traffic encrypted (HTTPS/TLS).

Role	Accessible Modules / Resources	Permission Level
Customer	Reservation (own: create, view), Menu (view), Payment (own: create)	Create own Reservation and Payment; read-only on Menu; no access to Orders, Tables, or administrative functions
Waiter	Table (assigned tables: view/update status), Reservation (view), Order (create/update), Payment (confirm)	Full operational access limited to assigned tables; cannot create Reservations or manage the Menu/Waiter assignments
Manager	Menu (full), Table (full), Waiter (assign), Reservation (view all), Order (view all), Payment (view all)	Full administrative access; read-only oversight of operational data — does not place orders or confirm payments

Reliability

No reservation, order, or payment data should be lost due to concurrent updates — for example, two waiters updating the same table at the same time.

Availability

The system should be available throughout restaurant operating hours, targeting 99.5% uptime.

Scalability

The data model should support a growing number of tables, waiters, and concurrent reservations without requiring redesign.

Maintainability

Because each module in Section 2.1 has a single responsibility (Section 1.3), individual modules can be modified or extended independently — for example, changing the allocation algorithm without touching Payment or Menu logic.

Usability

Waiter-facing screens should be optimized for fast, touch-based use on the restaurant floor, given the fast pace of service.

2.4 Entity Relationship (ER) Diagram

The diagram below illustrates the core entities from Section 2.1, their attributes, and their relationships, including primary keys (PK) and foreign keys (FK), with Reservation at the center as described in Section 1.2.

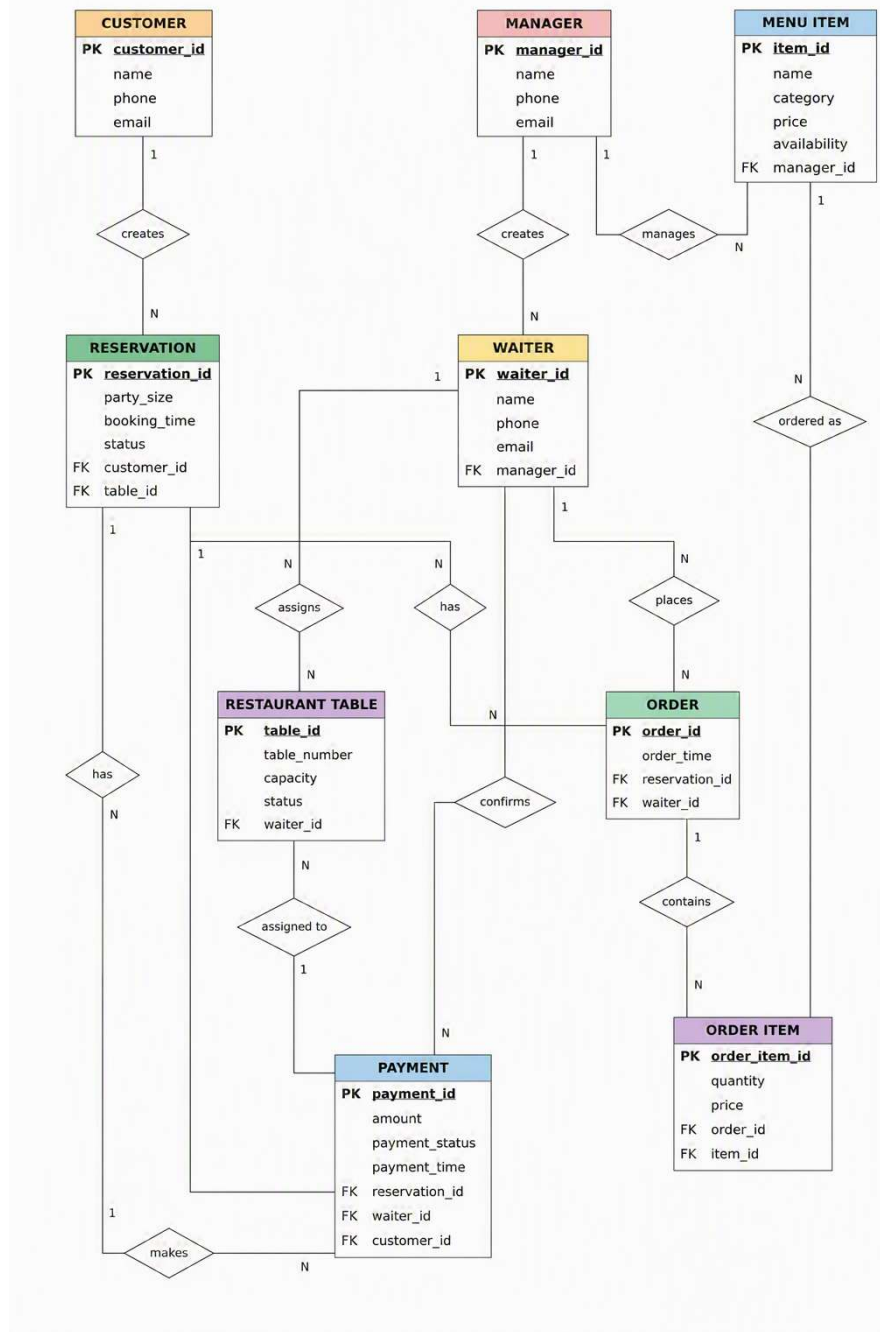


Figure 2: Entity Relationship Diagram — Restaurant Management System

2.5 Class Diagram

The class diagram translates the ER model into an object-oriented design, including a dedicated TableAllocationEngine service class that encapsulates the best-fit allocation logic from Section 2.1.4, keeping that business rule decoupled from the persistence classes..

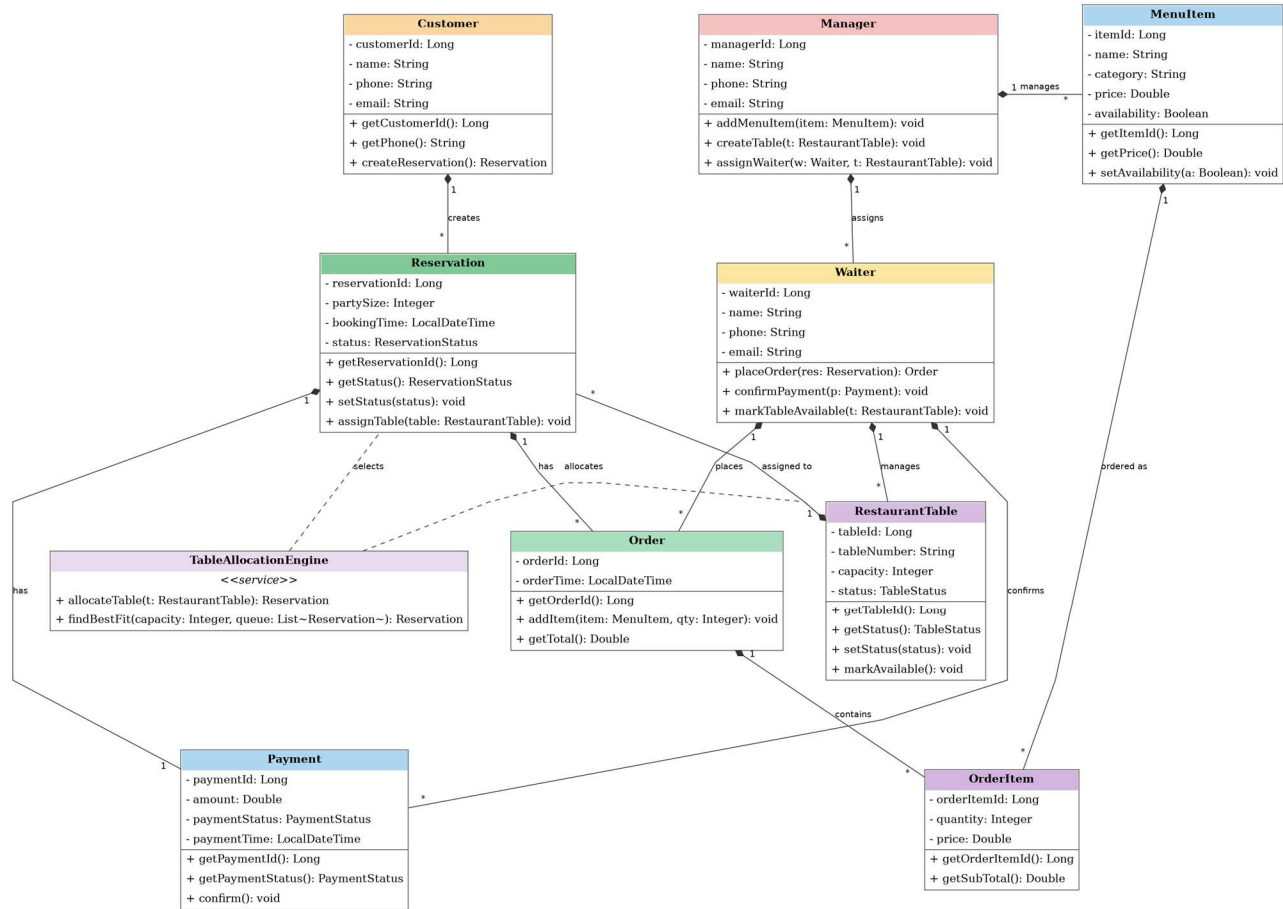


Figure 3: Class Diagram — Restaurant Management System

Chapter 3: Assumptions, Constraints and Acceptance Criteria

3.1 Assumptions and Constraints

- The system is designed for dine-in service only; delivery and takeaway are out of scope for this version.
- Each table is assigned to exactly one waiter, and each reservation is linked to at most one table, at any given time.
- The specific authentication technology remains an open item, to be finalized in the design phase (Section 2.3, Security).

3.2 Acceptance Criteria

- A reservation starts in WAITING status and automatically moves to SEATED once a best-fit table is allocated (Section 2.1.4).
- The allocation engine always selects the waiting reservation whose party size is the closest match, without exceeding capacity, when a table becomes available.
- A waiter can place multiple orders against a single SEATED reservation, each with multiple items.
- A reservation becomes COMPLETED only after the assigned waiter explicitly confirms payment.
- A table becomes AVAILABLE only after the waiter explicitly marks it so, which then triggers the next allocation.
- Every module in Section 2.1 enforces the role of restrictions defined in Section 2.3 (Security).
- The ER and Class diagrams (Sections 2.4–2.5) accurately reflect the implemented database schema and domain model at release.